CourtSy: technická dokumentace

Semestrální práce KTWA2 · Daniel Svoboda · říjen 2026 Repozitář `github.com/denysovski/ModernityRay` · Aplikace `denysovski.github.io/ModernityRay/`

CourtSy je frontendová aplikace fiktivního soukromého sportovního klubu. Umožňuje porovnat úrovně členství, rezervovat termín na kurtu, vybrat si tréninkový program, přihlásit se na klubovou akci, nakoupit v e-shopu a projít objednávkou. Backend neexistuje: obsah je uložený v datových modulech, stav aplikace v `localStorage`.

1. Framework a struktura projektu

Aplikace stojí na **Reactu 18** s nástrojem **Vite 5**. React jsem zvolil kvůli komponentovému modelu a kvůli hooku `useSyncExternalStore`, který řeší přesně to, co projekt potřebuje: napojení Reactu na stav uložený mimo něj. Vite kvůli rychlosti a bezkonfiguračnímu dělení kódu podle rout.

Projekt je členěný podle role souborů, ne podle jejich typu: `src/views/` obsahuje jednotlivé obrazovky, `src/layouts/` společný rám, `src/components/` dílčí prvky, `src/lib/` logiku bez Reactu, `src/data/` obsah webu a `src/routes.js` definici URL cest. Oddělení složky `lib/` je záměrné, protože nic v ní neimportuje React a dá se testovat bez vykreslování komponent.

TypeScript jsem nepoužil, protože ho zadání doporučuje, ale nevyžaduje. Datové struktury a veřejné funkce jsou místo toho otypované pomocí **JSDoc**, což dává v editoru stejné našeptávání.

---

2. Routování a architektura komponent

Routování zajišťuje **React Router 6** v režimu datového routeru (`createBrowserRouter`). Zvolil jsem ho místo jednoduchého `<BrowserRouter>` kvůli třem vlastnostem, které jinak nejsou k dispozici: obnova pozice scrollu, chybová hranice na úrovni routy a vlastnost `lazy` pro dělení kódu.

Aplikace má jednadvacet rout. Vzor „přehled a detail" se opakuje čtyřikrát (členství, areály, tréninkové programy, akce), což je stejná struktura nad různými daty a drží kód předvídatelný. K tomu patří úvodní stránka, cesty k členství (prohlídka, studentská sleva, dárkový poukaz), e-shop, restaurace, pokladna, účet, tři právní stránky a 404.

Tři právní stránky sdílejí jednu komponentu `Legal` a jeden balíček. Je to stejná osnova nad jiným textem, takže tři téměř shodné soubory by byly jen tři místa, která se musí držet v souladu; obsah je v `src/data/legal.js` a routa vybírá, který se vykreslí.

Strom komponent má tři vrstvy:

- `RootLayout` je namountovaný jednou nad `<Outlet />`. Navbar, košík ani patička se při navigaci neodmountují, což je smysl single page aplikace a důvod, proč obsah košíku přežije přechod mezi routami.
- **Views** si berou svůj výřez dat a vlastní lokální stav, například nastavení filtrů nebo vybranou fotografii v galerii.
- **Sdílené komponenty** jsou `PageHeader`, `Field` (formulářové pole včetně přístupnosti), `SmartLink` (rozliší routu od kotvy a od externího odkazu), `Img` (responzivní obrázek), `SelectMenu` (rozbalovací nabídka) a `ConfirmDialog` (potvrzení nevratné akce).

Poslední dvě nahrazují to, co prohlížeč nabízí sám, a stojí za zdůvodnění. Nabídku prvku `<select>` kreslí operační systém, takže nepřebírá zaoblení ani animaci zbytku aplikace; `window.confirm()` nelze stylovat vůbec a místo vysvětlení zobrazí adresu stránky. Vlastní komponenta ale musí vrátit vše, co vestavěný prvek uměl zadarmo: zavření klávesou Escape i kliknutím mimo, návrat fokusu na spouštěč, ovládání šipkami a klávesami Home/End, psaní prvních písmen u delšího seznamu a role `listbox` nebo `alertdialog` pro odečítače obrazovky. U dialogu je navíc fokus uvězněný uvnitř a po otevření sedí na tlačítku *Zrušit*, ne na tom nevratném.

Filtrování a řazení se počítají pomocí `useMemo` ze zdrojového pole, nejsou to kopie dat uložené ve stavu. Pro každý údaj tak existuje jediný zdroj pravdy.

---

3. Správa stavu a práce s daty

Stav je rozdělený podle toho, jestli má přežít znovunačtení stránky.

Trvalý stav (košík, profil, objednávky, nastavení) žije v modulu `src/lib/store.js` mimo React a ukládá se do `localStorage`. React se na něj napojuje pomocí `useSyncExternalStore` uvnitř `StoreContext`. Díky tomu je stav čitelný ještě před namountováním Reactu, takže počet položek v košíku je správný hned při prvním vykreslení. `useSyncExternalStore` je navíc hook přímo určený pro externí store a na rozdíl od dvojice `useEffect` a `useState` nemůže při souběžném renderování rozpadnout konzistenci. Práce s úložištěm je celá na jednom místě a žádná komponenta nesahá na `localStorage` přímo.

Redux jsem nepoužil záměrně. Celý stav tvoří jeden košík, jeden profil a tři přepínače, bez asynchronních operací. Context nad externím storem dává stejnou záruku jediného zdroje pravdy bez dalších patnácti kilobajtů kódu a zadání vlastní řešení výslovně

připouští. Rozhraním je hook `useStore()`, takže případná výměna implementace by se nedotkla ani jedné komponenty.

Dočasný stav, tedy otevřený košík, krok formuláře nebo nastavení filtrů, je v `useState` tam, kde se používá.

Odstranění dat má dvě úrovně a rozdíl mezi nimi je záměrný. `forgetMe()` smaže profil, košík a historii objednávek, ale ponechá nastavení zobrazení: smazat svoje data není totéž jako chtít, aby aplikace zapomněla, jak ji má člověk rád. `deleteAccount()` smaže i nastavení, tedy úplně vše, co si aplikace o uživateli drží. Obě akce se nejprve ptají přes `ConfirmDialog`.

Úložiště je ošetřené proti reálným situacím: každé čtení je obalené v `try/catch` kvůli privátnímu režimu a plné kvótě, uložená data se při načtení validují (ručně upravená položka košíku se zahodí, místo aby rozbila součty) a klíče jsou verzované, takže změna schématu stará data smaže místo toho, aby na nich aplikace spadla.

---

4. Validace vstupů a zabezpečení

Soubor `src/lib/validation.js` obsahuje čisté funkce ve tvaru „hodnota dovnitř, chybová hláška nebo `null` ven". Pokrývají e-mail, jméno, heslo, telefon, adresu, PSČ podle zvolené země, číslo karty, platnost a bezpečnostní kód. Číslo karty se kontroluje **Luhnovým algoritmem**, tedy stejnou kontrolou, jakou provádí každý reálný platební formulář. Z čísla se zároveň pozná vydavatel karty, což určuje očekávanou délku bezpečnostního kódu.

Pokladna validuje po jednotlivých krocích. Chyby se neukládají do stavu, ale odvozují při každém renderu, takže pole se opraví ve chvíli, kdy je hodnota platná. Neplatné pole dostane atribut `aria-invalid`, hlášku s `role="alert"` a focus skočí na první chybu.

Frontend nemůže nahradit kontrolu na serveru a netváří se tak. Dělá tři věci, které skutečně patří klientovi:

- **Neukládá nic tajného.** Heslo se zkontroluje a zahodí, z čísla karty zůstanou pouze poslední čtyři číslice a název vydavatele.
- **Sanitizuje před uložením.** Funkce `sanitizeText()` omezí délku a odstraní řídicí znaky, takže nelze nafouknout úložiště ani poškodit uložený JSON.
- **Odmítne nemožný vstup.** Neplatný kontrolní součet karty nebo expirace v minulosti se do stavu aplikace vůbec nedostanou.

Nikde v aplikaci není použit `dangerouslySetInnerHTML`, protože React escapuje veškerý vykreslovaný text sám.

---

5. Testování

Vrstva	Nástroj	Počet	Co ověřuje
Unit	Vitest	87	validace, store a součty, formátování dat
Komponenty	Vitest + React Testing Library	15	přístupnost formulářového pole a rozbalovací nabídky, průchod pokladnou
E2E	Playwright (Chromium)	39	routování, nákupní cesta, nastavení, mazání účtu, stránka 404

Jednotkové testy míří na složku `lib/`, kde je logika, která se může tiše pokazit: hraniční data v kalendáři, Luhnův algoritmus, PSČ podle zemí nebo chování storu nad poškozeným `localStorage`. Funkce, které potřebují aktuální datum, ho dostávají jako parametr, takže testy nečtou systémový čas a budou platit i příští rok.

End to end testy běží proti produkčnímu buildu, protože jen tak se odhalí chyba, která vznikne až rozdělením kódu na samostatné balíčky. Nejdelší test projde celou cestu: výběr členství, rezervace kurtu, čtyři kroky pokladny včetně zamítnutých vstupů, platba a nalezení objednávky v účtu po znovunačtení stránky.

Jeden test hlídá vlastnost, kterou by jinak nikdo nezpozoroval, dokud by nebylo pozdě: že aplikace je pořád **single page**. Projde kliknutím všechny routy a počítá odpovědi typu dokument. Musí zůstat u jedné a hodnota zapsaná do `window` na začátku musí přežít až do konce i po tlačítkách zpět a vpřed. Tahle vlastnost se totiž ztrácí po jednom odkazu: stačí jeden `<a href>`, který router nespravuje, a daná routa začne tiše načítat stránku znovu, zatímco navenek vypadá všechno správně. Porovnávání adres by to neodhalilo, počítání dokumentů ano.

---

6. Profilování a optimalizace

Měřeno nástrojem Lighthouse nad produkčním buildem v režimu mobilní emulace. Uložené reporty jsou ve složce `docs/lighthouse/`, měření lze zopakovat příkazem `npm run audit`.

Routa	Výkon	Přístupnost	LCP	CLS
`/`	70 → **89**	90 → **100**	9,8 s → **3,4 s**	0 → **0**
`/memberships`	68 → **92**	94 → **100**	3,2 s → **3,0 s**	0,58 → **0**
`/courts/centre-court`	57 → **91**	96 → **100**	5,0 s → **3,2 s**	0,59 → **0**

Kategorie Best Practices a SEO jsou po úpravách na hodnotě **100** na všech měřených routách. Co měření odhalilo:

- **Posun layoutu (CLS) 0,58.** Dělení kódu pomocí `React.lazy` a `<Suspense>` vykreslilo nejdřív rám se zástupným prvkem a teprve potom obsah, čímž posunulo patičku přes půl stránky. Přechod na `lazy` na úrovni routy to vyřešil: router počká na načtení modulu a vykreslí stránku najednou. Stejný počet balíčků, stejný objem dat, nulový posun.
- **Vykreslení hlavního obsahu (LCP) 9,8 s.** Fotografie byly soubory JPEG o šířce 1600 px a velikosti 200 až 550 kB. Skript `scripts/responsive-images.mjs` z nich pomocí knihovny Sharp generuje WebP ve čtyřech šířkách a komponenta `Img` z nich staví prvek `<picture>`. Hlavní obrázek úvodní stránky klesl z 293 kB na 72 kB.
- **Blokující požadavek při vykreslování.** Písmo Inter se stahovalo z cizí domény. Nyní je součástí buildu, takže na kritické cestě není.
- **Přístupnost.** Kontrast drobných textů nesplňoval úroveň WCAG AA, pořadí nadpisů přeskakovalo z `h2` na `h4`, tečky karuselu měly 8 px místo minimálních 24 px a sbalené mobilní menu nechávalo zhruba dvacet odkazů v pořadí tabulátoru. Vše je opravené.
- **Omezení pohybu.** `MotionConfig` respektuje systémové nastavení, jenže řídí pouze Framer Motion. Přechody v CSS, běžící text v patičce a posouvání karuselu o něm nevědí, takže volba *Omezit pohyb* fungovala jen z poloviny. Nastavení proto zapisuje atribut `data-reduced-motion` na element `<html>` a jedno pravidlo v CSS zkrátí všechny přechody i animace na setinu milisekundy — stejně, jako to dělá dotaz `prefers-reduced-motion`, který v projektu do té doby také chyběl. Animace se nevypínají úplně: zkrácení zachová případné události `animationend` a nechá prvek v koncovém, ne v prvním snímku.

Dělení kódu jsem zachoval, i když u prvního načtení mnoho dat neušetří: sdílené balíčky `react` a `framer-motion` se mezi nasazeními nemění, takže vracejícímu se návštěvníkovi zůstanou v mezipaměti prohlížeče.